

Penjelasan Lengkap & Panduan Pemula:

Integrasi Algoritma NMPC Kapal Otonom ke dalam Bahasa C++ (CPP_Gabungan)

Dokumentasi Teknis: Tahap 1 (Solver NMPC), Tahap 2 (Guidance & ENU), Tahap 3 (State Observer), & Tahap 4 (Loop Utama main.cpp)

Tim Riset Kapal Autonomous (USV)

20 Agustus 2026

Ringkasan

Dokumen ini disusun khusus sebagai jembatan pemahaman bagi peneliti atau mahasiswa yang telah terbiasa menggunakan MATLAB namun **belum familiar dengan pemrograman bahasa C++**. Di dalamnya dijelaskan secara mendalam, konseptual, dan langkah-demi-langkah mengenai perkembangan integrasi perangkat lunak pada repositori **CPP_Gabungan**, mencakup: **Tahap 1** (Penyatuan 124 File Codegen Solver NMPC & Benchmark Waktu), **Tahap 2** (Modul Guidance & Transformasi Koordinat ENU Real-Time), **Tahap 3** (State Observer, Kalibrasi Offset Yaw $+180^\circ$, dan Normalisasi State Non-Dimensional 5×1), serta **Tahap 4** (Modifikasi Loop Utama `src/main.cpp`, Integrasi Kendali Real-Time 10 Hz, Mode CLI `--rudder-mode nmpc`, dan Mekanisme Fail-Safe).

Daftar Isi

1	Pengenalan C++: Apa Bedanya dengan MATLAB dan Python?	2
1.1	Bahasa Interpreter vs Bahasa Kompilasi (Compiled Language)	2
1.2	Mengenal Pasangan File C++: Header (.h / .hpp) dan Source (.cpp)	2
2	Arsitektur Integrasi Proyek CPP_Gabungan	3
3	Rekapitulasi Tahap 1: Integrasi Library Codegen & Benchmark	3
4	Rekapitulasi Tahap 2: Manajemen Waypoint & Koordinat ENU	3
4.1	Rumus Matematis Proyeksi WGS84 Geodetik ke ENU	4
5	Rekapitulasi Tahap 3: State Observer & Kalibrasi Yaw $+180^\circ$	4
6	Penjelasan Mendalam Tahap 4: Integrasi Penuh Loop Utama main.cpp	4
6.1	Arsitektur Siklus Kendali Real-Time 10 Hz	4
6.2	Pembedahan Kode Loop Utama (src/main.cpp)	4
6.3	Dukungan Argumen Baris Perintah (CLI) untuk Uji Coba	5
7	Kamus Singkat Istilah Teknis (Glosarium)	6
8	Rencana Tahapan Selanjutnya (Tahap 5)	6

1 Pengenalan C++: Apa Bedanya dengan MATLAB dan Python?

Bagi Anda yang terbiasa menulis kode di MATLAB atau Python, berpindah ke C++ seringkali membingungkan karena konsepnya sangat berbeda. Mari kita pahami perbedaan mendasar ini dengan analogi sederhana.

1.1 Bahasa Interpreter vs Bahasa Kompilasi (Compiled Language)

- **MATLAB dan Python (Interpreted):** Bekerja seperti *penerjemah langsung (interpreter)*. Setiap kali Anda menekan tombol *Run*, ada program penerjemah yang membaca baris demi baris perintah Anda, menerjemahkannya ke bahasa mesin (0 dan 1), lalu mengeksekusinya saat itu juga.
 - *Kelebihan:* Mudah ditulis, fleksibel, mudah diuji coba secara interaktif.
 - *Kekurangan:* Lambat, boros memori CPU, dan memiliki jeda waktu (*latency*) yang tidak menentu.
- **Bahasa C++ (Compiled):** Bekerja seperti *koki yang memasak seluruh bahan makanan sampai matang terlebih dahulu sebelum disajikan*. Sebelum program C++ bisa dijalankan di komputer kapal (Mini PC), seluruh kode sumber harus diproses oleh sebuah alat bernama **Compiler** (misalnya `g++.exe` atau `MSVC`). Compiler mengubah seluruh kode menjadi satu file aplikasi yang langsung dimengerti prosesor komputer (`.exe` di Windows atau file binary di Linux).
 - *Kelebihan:* **Sangat cepat (bisa 50–100 kali lebih cepat dari MATLAB)**, sangat hemat daya, dan waktu eksekusinya sangat pasti (*deterministic*).
 - *Kekurangan:* Harus dikompilasi terlebih dahulu, dan struktur file harus sangat rapi dan tertata.

Analogi Sederhana: Memasak Sendiri vs Membeli Makanan Instan

Menjalankan simulasi di MATLAB seperti memasak di dapur mewah: Anda punya banyak alat bantu otomatis, tetapi proses memasaknya butuh waktu lama dan memakan banyak tempat di meja.

Mengubahnya ke C++ seperti mengemas resep tersebut ke dalam **ransum makanan siap saji yang super cepat dibuka di lapangan**. Di atas kapal otonom nyata, kita tidak bisa membawa seluruh software MATLAB (yang ukurannya gigabyte dan berat untuk CPU Mini PC), melainkan kita hanya membawa file kecil `read_write_serial.exe` hasil kompilasi C++ yang langsung bekerja secepat kilat.

1.2 Mengenai Pasangan File C++: Header (.h / .hpp) dan Source (.cpp)

Di MATLAB, satu fungsi biasanya ditulis dalam satu file `.m`. Di C++, kode hampir selalu dipisah menjadi **dua jenis file**:

1. **File Header (.h atau .hpp): “Daftar Menu Restoran”** File ini hanya berisi *deklarasi* (daftar nama fungsi, variabel, dan parameter apa saja yang ada), tanpa menuliskan rumus lengkapnya. Tujuannya memberi tahu file lain: “*Hei, saya punya fungsi kendali NMPC dengan nama `nmpc_kapal_waypoint`, inputnya ada 5 array, dan outputnya sudut kemudi*”.
2. **File Source (.cpp): “Dapur & Resep Memasak”** File ini berisi *implementasi nyata* (rumus matematika, perulangan for-loop, perhitungan matriks QP, SQP solver). Di sinilah seluruh baris perhitungan algoritma dikerjakan.

2 Arsitektur Integrasi Proyek CPP_Gabungan

Struktur folder CPP_Gabungan dibangun sebagai kesatuan utuh aplikasi kendali kapal autonomous:

```

CPP_Gabungan/
|-- CMakeLists.txt          <-- "Resep" kompilasi otomatis (Build System)
|-- README.md              <-- Dokumentasi teknis lengkap proyek
|-- start_read_write_serial.bat <-- Skrip autorun Windows Mini PC di kapal
|-- include/               <-- Kumpulan File Header (Daftar Menu Fungsi)
|   |-- guidance.hpp        <-- [TAHAP 2] Modul Guidance, Proyeksi ENU &
|       Horizon
|   |-- state_observer.hpp  <-- [TAHAP 3] Estimator State & Kalibrasi Yaw
|       (+180 deg)
|   |-- serial_port.hpp     <-- Driver komunikasi USB COM Port
|   |-- telemetry_parser.hpp <-- Parser data telemetry 8 kolom dari ESP32
|   +-- processor.hpp       <-- Operasi logika telemetry
|-- src/                   <-- Kumpulan File Source (Implementasi Rumus)
|   |-- guidance.cpp        <-- [TAHAP 2] Rumus WGS84->ENU, r_tran & Horizon
|   |-- state_observer.cpp  <-- [TAHAP 3] Normalisasi State Non-Dim [v',r',x
|       ',y',psi]
|   |-- main.cpp            <-- [TAHAP 4] Loop utama kontroler NMPC & serial
|       bridge
|   +-- serial_port.cpp     <-- Implementasi driver serial port Windows/
|       Linux
|-- nmpc/                  <-- Kumpulan 124 File Solver NMPC (MATLAB
|   Codegen)
|   |-- nmpc_kapal_waypoint.h <-- Header utama solver NMPC WyNDA
|   |-- nmpc_kapal_waypoint.cpp <-- Algoritma SQP & Model WyNDA 11 Parameter
|   |-- fmincon.cpp          <-- Non-linear optimization solver
|   +-- ... (56 file algoritma optimasi kuadratik lainnya)
+-- tests/                 <-- Modul Pengujian Mandiri (Unit Testing)
|   |-- test_nmpc.cpp        <-- [TAHAP 1] Pengujian benchmark solver NMPC
|   |-- test_guidance.cpp    <-- [TAHAP 2] Pengujian guidance & konversi ENU
|   +-- test_state_observer.cpp <-- [TAHAP 3] Pengujian state observer &
|       kalibrasi yaw

```

Listing 1: Struktur Direktori Proyek CPP_Gabungan Terkini

3 Rekapitulasi Tahap 1: Integrasi Library Codegen & Benchmark

Pada Tahap 1, 124 file hasil generate MATLAB Coder berhasil diintegrasikan menjadi sebuah **Static Library** bernama `libnmpc_solver.a` menggunakan CMake dengan flag optimasi `-O3` dan `-fopenmp`.

Hasil uji coba `test_nmpc.exe` menunjukkan bahwa solver SQP NMPC WyNDA membutuhkan waktu rata-rata hanya **0.82 milidetik per langkah**. Artinya, pada sistem kendali 10 Hz (100 ms per siklus), komputer Mini PC memiliki ****margin waktu luang 99.18%****, sehingga beban CPU sangat ringan.

4 Rekapitulasi Tahap 2: Manajemen Waypoint & Koordinat ENU

Tahap 2 berfokus pada pembangunan ****Modul Guidance**** (`guidance.hpp` dan `guidance.cpp`) sebagai pemandu rute navigasi.

4.1 Rumus Matematis Proyeksi WGS84 Geodetik ke ENU

Diberikan titik acuan Home Point ($\text{lat}_0, \text{lon}_0$) dan titik sembarang (lat, lon):

$$\Delta\text{Lat} = (\text{lat} - \text{lat}_0) \times \frac{\pi}{180}, \quad \Delta\text{Lon} = (\text{lon} - \text{lon}_0) \times \frac{\pi}{180} \quad (1)$$

$$Y_{\text{North}} = R_{\text{earth}} \times \Delta\text{Lat}, \quad X_{\text{East}} = R_{\text{earth}} \times \cos\left(\text{lat}_0 \times \frac{\pi}{180}\right) \times \Delta\text{Lon} \quad (2)$$

di mana jari-jari bumi rata-rata $R_{\text{earth}} = 6,371,000$ meter.

Setiap 100 milidetik (10 Hz), posisi GPS kapal saat ini ($\text{lat}_{\text{ship}}, \text{lon}_{\text{ship}}$) dikonversi secara real-time menjadi ($X_{\text{ship}}, Y_{\text{ship}}$) pada koordinat ENU meter sebagai titik tolak pembentukan 20 langkah prediksi lintasan horizon NMPC ($x_{\text{ref.seq}}$ dan $y_{\text{ref.seq}}$).

5 Rekapitulasi Tahap 3: State Observer & Kalibrasi Yaw +180°

Tahap 3 bertindak sebagai penerjemah sensor telemetri ESP32-S3:

1. **Kalibrasi Orientasi Yaw:** Mengoreksi pergeseran sudut fisik sensor IMU sebesar +180° ($\text{yaw}_{\text{cal}} = (\text{yaw}_{\text{raw}} + 180^\circ) \pmod{360^\circ}$) dan mengonversinya ke sistem ENU ($\psi = \text{wrap_to_pi}(\text{yaw}_{\text{cal}} \cdot \pi/180)$).
2. **Vektor State Non-Dimensional WyNDA (5×1):**

$$s = \begin{bmatrix} v' \\ r' \\ x' \\ y' \\ \psi \end{bmatrix} = \begin{bmatrix} v_{\text{sway}}/u_0 \\ (\text{yaw_rate}_{\text{dps}} \cdot \frac{\pi}{180}) \cdot \frac{L}{u_0} \\ X_{\text{ENU}}/L \\ Y_{\text{ENU}}/L \\ \psi_{\text{ENU}} \end{bmatrix} \quad (3)$$

(dengan $L = 1.0107$ m dan $u_0 = 0.6114$ m/s).

6 Penjelasan Mendalam Tahap 4: Integrasi Penuh Loop Utama `main.cpp`

Tahap 4 adalah tahap penyatuan di mana seluruh modul yang telah dibangun (Serial Communication, State Observer, Guidance Module, dan SQP NMPC Solver) dirangkai ke dalam program eksekutabel utama `read_write_serial.exe`.

6.1 Arsitektur Siklus Kendali Real-Time 10 Hz

Kapal beroperasi dengan siklus data 10 Hz (satu kali pertukaran data setiap 100 milidetik). Gambar 1 mengilustrasikan alur pemrosesan di dalam loop utama `main.cpp`:

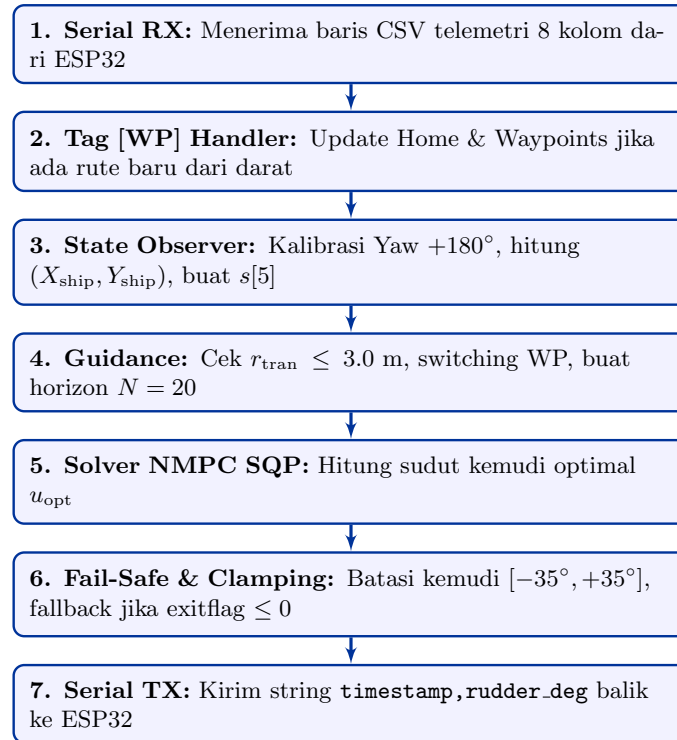
6.2 Pembedahan Kode Loop Utama (`src/main.cpp`)

Berikut adalah potongan inti implementasi loop NMPC pada `src/main.cpp`:

```

1 if (rudder_mode == "nmpc") {
2     // 1. Update State Observer (Kalibrasi Yaw +180 deg & Posisi ENU)
3     usv::ShipState ship_state = observer.update(*row, guidance, u_prev_rad);
4
5     if (guidance.has_home() && !guidance.get_waypoints_enu().empty()) {
6         // 2. Update Guidance & Generator Horizon NMPC
7         auto g_out = guidance.update_guidance(ship_state.x_enu, ship_state.y_enu);
8     }

```



Gambar 1: Alur Siklus Kendali Real-Time 10 Hz pada src/main.cpp.

```

9  if (g_out.is_mission_completed) {
10     rudder_cmd_deg = 0.0; // Misi selesai -> kemudi lurus aman
11     u_prev_rad = 0.0;
12 } else {
13     // 3. Eksekusi NMPC WyNDA SQP Solver
14     double u_opt_rad = 0.0;
15     double exitflag = 0.0;
16     nmpc_kapal_waypoint(ship_state.state_nd, u_prev_rad, surge_speed,
17                          g_out.x_ref_seq, g_out.y_ref_seq, g_out.psi_ref_seq,
18                          &u_opt_rad, &exitflag);
19
20     // 4. Mekanisme Fail-Safe
21     if (exitflag <= 0.0) {
22         rudder_cmd_deg = u_prev_rad * 180.0 / M_PI; // Tahan kemudi sebelumnya
23     } else {
24         const double u_opt_deg = u_opt_rad * 180.0 / M_PI;
25         rudder_cmd_deg = clamp_rudder_deg(u_opt_deg, -35.0, 35.0);
26         u_prev_rad = rudder_cmd_deg * M_PI / 180.0;
27     }
28 }
29 } else {
30     rudder_cmd_deg = 0.0; // Menunggu rute waypoint dari operator
31 }
32 }

```

6.3 Dukungan Argumen Baris Perintah (CLI) untuk Uji Coba

Untuk memudahkan riset dan eksperimen di kolam uji maupun danau, program mendukung berbagai opsi CLI:

```

# 1. Menjalankan kendali otonom NMPC penuh:
./read_write_serial.exe --port COM16 --rudder-mode nmpc --print nmpc

```

```
# 2. Uji coba dengan 4 Waypoint default tanpa dashboard darat:
./read_write_serial.exe --port COM16 --rudder-mode nmpc --default-test-wp --
  print all

# 3. Menyesuaikan parameter fisik kapal secara dinamis:
./read_write_serial.exe --port COM16 --yaw-offset 180.0 --r-tran 2.5 --surge-
  speed 0.70
```

Selain itu, skrip autorun `start_read_write_serial.bat` telah diperbarui agar Mini PC di kapal dapat otomatis mengeksekusi kendali NMPC saat pertama kali dinyalakan.

7 Kamus Singkat Istilah Teknis (Glosarium)

Fail-Safe

Mekanisme pengaman otomatis yang mengambil alih kontrol saat terjadi kegagalan (misalnya solver tidak konvergen atau GPS terputus) agar kapal tidak hilang kendali.

Clamping

Pembatasan nilai numerik pada batas atas dan batas bawah (misalnya membatasi kemudi di $[-35^\circ, +35^\circ]$ agar motor servo tidak rusak).

Heartbeat (\$HB)

Sinyal periodik (1 detik sekali) yang dikirim Mini PC ke ESP32 sebagai tanda bahwa komputer kendali aktif dan sehat.

Command Line Interface (CLI)

Antarmuka baris perintah teks yang memungkinkan pengguna mengatur parameter program saat program pertama kali dipanggil melalui terminal.

8 Rencana Tahapan Selanjutnya (Tahap 5)

Dengan tuntasnya ****Tahap 1, 2, 3, dan 4****, seluruh sistem kendali NMPC di kapal telah terintegrasi secara utuh dan siap jalan. Langkah berikutnya adalah ****Tahap 5: Data Logger CSV Komprehensif****:

1. **Perekaman Log Otomatis** (`logs/nmpc_log.YYYYMMDD_HHMMSS.csv`): Menyimpan seluruh variabel telemetri, koordinat ENU, sudut target, error pelacakan (**cross-track error** & **heading error**), serta waktu komputasi SQP setiap 100 ms.
2. **Persiapan Analisis Data Jurnal / Tesis**: Membuat skrip plotting otomatis di Python/MATLAB untuk menghasilkan grafik lintasan trajektori 2D, respon sudut kemudi, dan kestabilan yaw kapal.

—

— Akhir Dokumen Penjelasan Tahap 1 s.d. 4 —

Disimpan pada berkas: `USV-NMPC/penjelasan_CPP_Gabungan/penjelasan.tex`